“‘{r debug} getwd()

list.files()

list.files(“figures”)

file.exists(“figures/prithvi_embeddings.png”)

file.exists(“figures/prithvi_cosine.png”)

```
# Prithvi

## Introduction

In this tutorial, we implement a complete pipeline for extracting geospatial embeddings using the **Pri

The original Prithvi repository relies on OpenMMLab components such as **MMCV** and **MMSegmentation** 

The workflow is:

‘‘‘text
Google Earth Engine
-> Sentinel-2 GeoTIFF
-> Image preprocessing
-> TemporalViTEncoder
-> CLS Token
-> 1024-dimensional embedding
```

Exporting Sentinel-2 imagery from Google Earth Engine

The satellite imagery was obtained using **Google Earth Engine** and the **COPERNICUS/S2_SR_HARMONIZED** collection.

Five Mexican cities were selected:

- Mexico City
- Guadalajara
- Monterrey
- Merida
- Oaxaca

For each city, a Sentinel-2 image patch was exported as a GeoTIFF using six spectral bands.

Band	Description
B2	Blue
B3	Green
B4	Red
B8	Near Infrared (NIR)
B11	Short-Wave Infrared 1 (SWIR1)
B12	Short-Wave Infrared 2 (SWIR2)

These bands contain information about vegetation, urban areas, moisture, and other physical characteristics of the Earth’s surface.

Image preprocessing

Each GeoTIFF image was loaded using **rasterio**.

Initially, every image has the following shape:

```
(bands, height, width)
```

For example, the Mexico City image was loaded as:

```
(6, 225, 237)
```

Since the encoder expects images of size **224 x 224 pixels**, each patch was resized using bilinear interpolation.

Pixel values were normalized by dividing by **10000**, following the usual preprocessing for Sentinel-2 surface reflectance products.

Finally, the tensor was rearranged into the format expected by Prithvi:

```
(batch, channels, time, height, width)
```

resulting in:

```
(1, 6, 1, 224, 224)
```

Embedding extraction

The preprocessed tensor was passed through the **TemporalViTEncoder**.

The encoder divides the image into patches, converts each patch into a token, and processes the complete sequence using Transformer blocks.

The encoder output has shape:

```
(1, 197, 1024)
```

where:

- **1** is the batch size.
- **197** corresponds to 196 image patches plus one CLS token.
- **1024** is the embedding dimension of each token.

The global image representation is obtained by selecting the CLS token:

```
(1, 1024)
```

Thus, every city is represented by a single embedding vector:

$$z_i \in \mathbb{R}^{1024}.$$

Generated outputs

The same procedure was applied to the five selected cities.

The pipeline produced five NumPy embedding files and one consolidated CSV file:

```
outputs/

- cdmx_prithvi_embedding.npy
- guadalajara_prithvi_embedding.npy
- monterrey_prithvi_embedding.npy
- merida_prithvi_embedding.npy
- oaxaca_prithvi_embedding.npy
- prithvi_mexico_embeddings.csv
```

The CSV file contains one row per city and 1024 columns corresponding to the embedding dimensions.

PCA visualization

Principal Component Analysis (PCA) was applied to project the original 1024-dimensional embeddings into two dimensions.

The PCA projection shows that each city occupies a different position in the latent space learned by Prithvi.

Although only five cities were analyzed, the encoder produces distinct representations for each location. This suggests that it captures spatial and spectral characteristics from Sentinel-2 imagery.

For example, Merida appears separated from the remaining cities along the first principal component, while Guadalajara and Monterrey are located relatively close to each other.

Cosine similarity analysis

Cosine similarity was computed between every pair of embeddings.

All pairwise similarities are above **0.97**, indicating that the embeddings share a common latent structure.

This is expected because all embeddings were extracted from Sentinel-2 imagery using the same pretrained foundation model.

However, the similarities are not identical. Guadalajara and Monterrey show the highest similarity, while Merida shows slightly lower similarity with the remaining cities. This suggests that the model captures meaningful geographic differences.

Interpretation

This experiment shows that Prithvi can be used as a general-purpose geospatial embedding extractor, even though it was originally designed as a backbone for segmentation models.

The experiment verifies that:

1. Google Earth Engine provides Sentinel-2 imagery compatible with Prithvi.
2. The exported GeoTIFF images can be adapted to the required input format through preprocessing.
3. The TemporalViTEncoder produces a latent representation composed of 197 tokens.
4. The CLS token provides a compact global representation of the image.
5. Every city is represented by a 1024-dimensional embedding.
6. The extracted embeddings show measurable differences through PCA and cosine similarity.

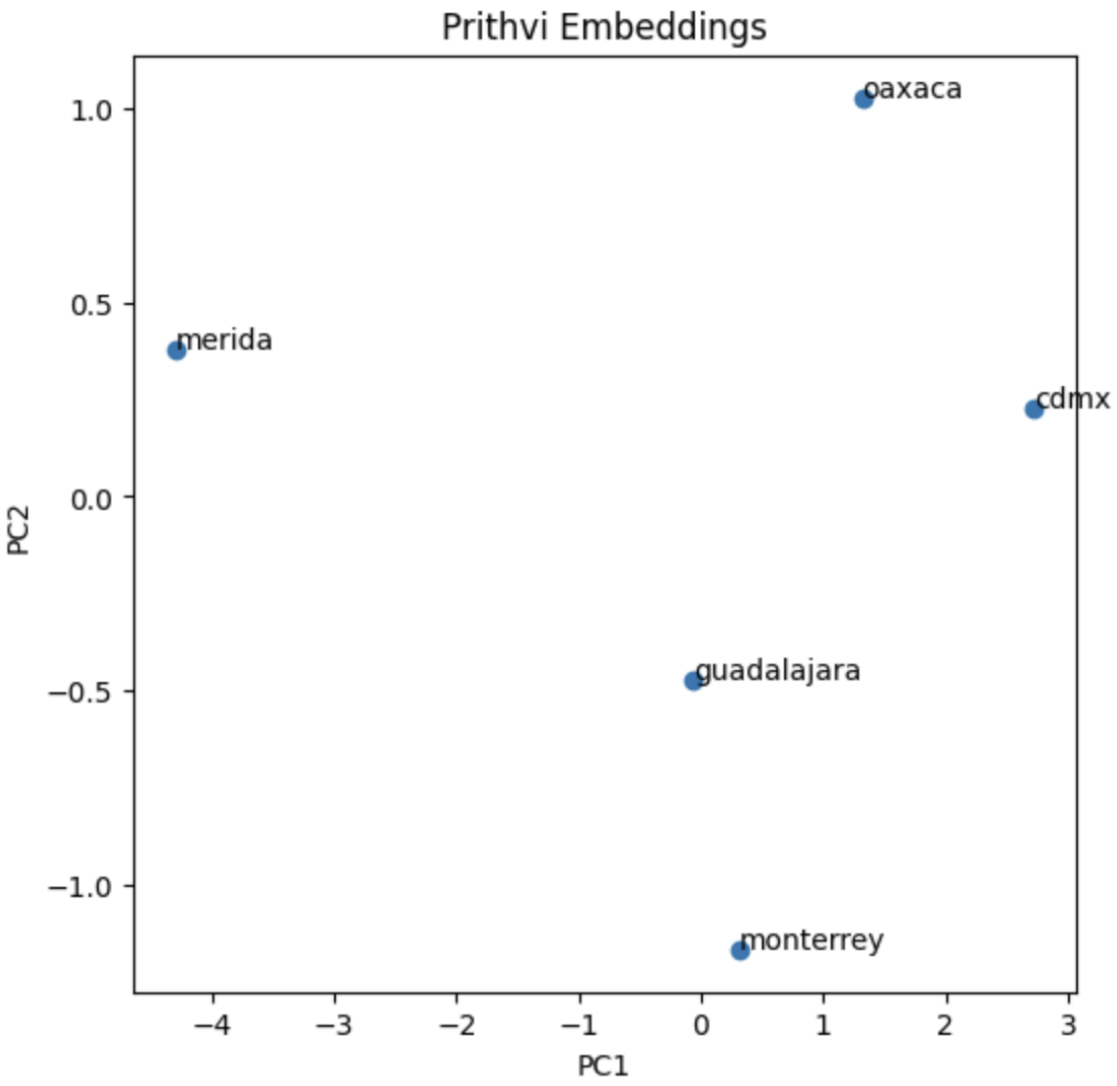

Figure 1: PCA Embeddings

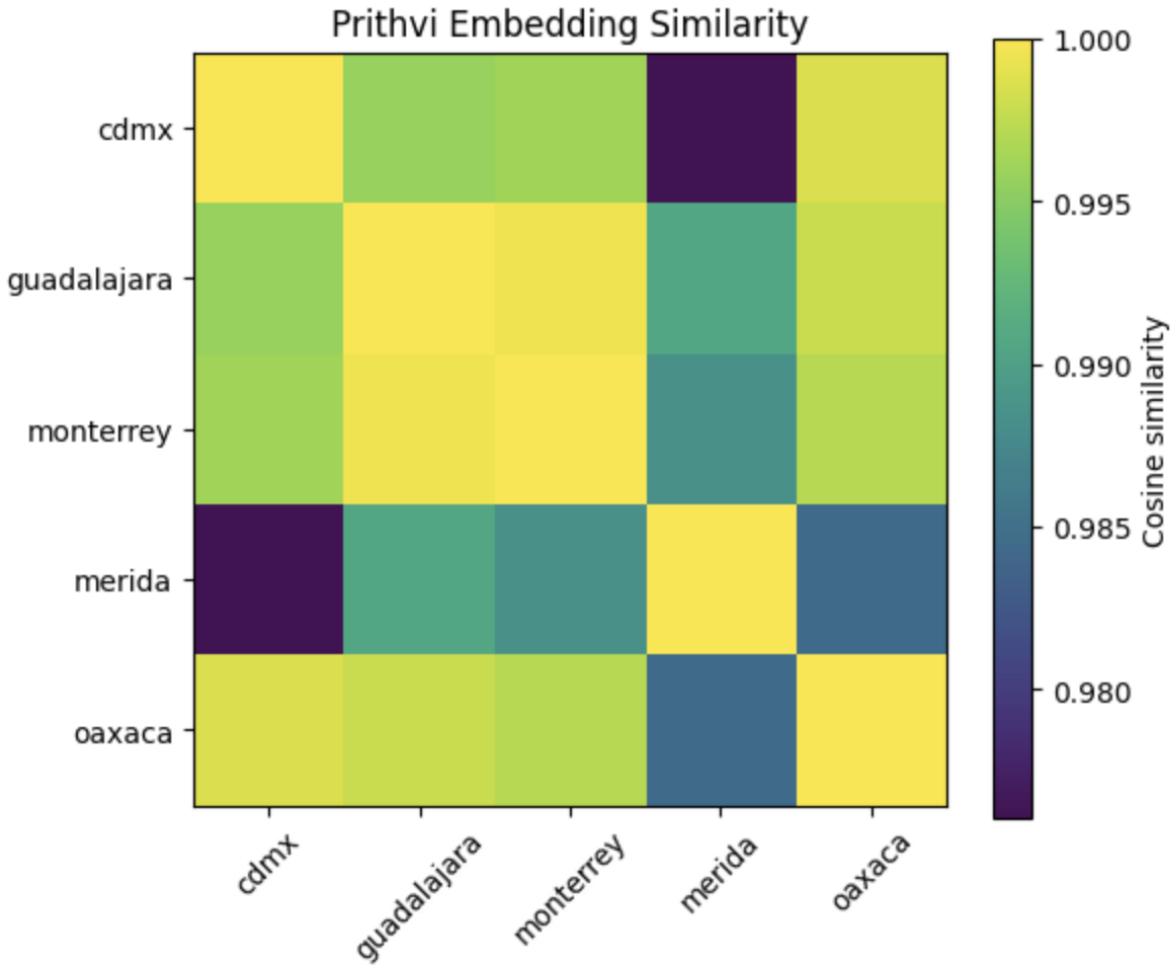

Figure 2: Cosine Similarity

Comparison with Clay

This implementation complements the previous experiment with **Clay**.

Both models generate **1024-dimensional embeddings**, but they were designed with different objectives.

Clay directly outputs geospatial embeddings intended for downstream analysis, whereas Prithvi was originally designed as a Transformer backbone for downstream tasks such as segmentation and classification. Therefore, in this implementation, the encoder was isolated and the CLS token was manually extracted.

Model	Input	Architecture	Output representation	Dimension
Clay	Satellite imagery	Geospatial foundation model	Direct embedding	1024
Prithvi	Sentinel-2 imagery	Temporal Vision Transformer	CLS token	1024

Conclusion

A complete pipeline for extracting geospatial embeddings using the Prithvi Foundation Model was successfully implemented.

Each city is represented by a 1024-dimensional vector summarizing the spectral and spatial information contained in its Sentinel-2 image patch.

This experiment demonstrates that Earth observation foundation models can be used not only for supervised tasks such as semantic segmentation, but also as general-purpose feature extractors.

The generated embeddings provide a baseline for future comparisons with other foundation models, especially **AlphaEarth Foundations**, in terms of embedding dimensionality, latent-space structure, implementation complexity, and geospatial representation quality.